10
Graph Algorithms

The graph representations and traversal algorithms of Chapter 9 provide the basic building blocks for any computation on graph structures. In this chapter, we consider more advanced graph theory and algorithms.

Graph theory is the study of the properties of graph structures. It provides us with a language with which to talk about graphs. The key to solving many problems is identifying the fundamental graph-theoretic notion underlying the situation and then using classical algorithms to solve the resulting problem.

We begin with a overview of basic graph theory and follow with algorithms for finding important structures such as minimum spanning trees, shortest paths, and maximum flows.

10.1 Graph Theory

In this section, we provide a quick review of basic graph theory. Several excellent books on graph theory [PS03, Wes00] are available for more detailed information. We outline relevant algorithms which should be fairly simple to program given the machinery developed in the previous chapter.

10.1.1 Degree Properties

Graphs are made up of vertices and edges. The simplest property of a vertex is its *degree*, the number of edges incident upon it.

There are several important properties of vertex degrees. The sum of the vertex degrees in any undirected graph is twice the number of edges, since every edge contributes one to the degree of both adjacent vertices. A corollary to this is that every graph contains an even number of odd degree vertices. For directed graphs, the relevant degree condition is that the sum of the in-degrees of all vertices equals the sum of all out-degrees. The parity of vertex degrees has an important role in recognizing Eulerian cycles as discussed in Section 10.1.3.

Trees are undirected graphs which contain no cycles. Vertex degrees are important in the analysis of trees. A *leaf* of a tree is a vertex of degree 1. Every n-vertex tree contains $n-1$ edges, so all non-trivial trees contain at least two leaf vertices. Deleting a leaf leaves a smaller tree, trimming the tree instead of disconnecting it.

Rooted trees are directed graphs where every node except the root has in-degree 1. The leaves are the nodes with out-degree 0. *Binary trees* are rooted trees where every vertex has an out-degree of 0 or 2. At least half the vertices of all such binary trees must be leaves.

A *spanning tree* of a graph $G = (V, E)$ is a subset of edges $E' \subset E$ such that E' is a tree on V. Spanning trees exist for any connected graph; the parent relation encoding vertex discovery for either breadth-first or depth-first search suffices to construct one. The *minimum spanning tree* is an important property of weighted graphs, and discussed in Section 10.2.

10.1.2 Connectivity

A graph is *connected* if there is an undirected path between every pair of vertices. The existence of a spanning tree is sufficient to prove connectivity. A depth-first search-based connected components algorithm was presented in Section 9.4.2.

However, there are other notions of connectivity to be aware of. The *vertex (edge) connectivity* is the smallest number of vertices (edges) which must be deleted to disconnect the graph. The most interesting special case when there is a single weak link in the graph. A single vertex whose deletion disconnects the graph is called an *articulation vertex*; any graph without such a vertex is said to be *biconnected*. A single edge whose deletion disconnects the graph is called a *bridge*; any graph without such an edge is said to be *edge-biconnected*.

Testing for articulation vertices or bridges is easy via brute force. For each vertex/edge, delete it from the graph and test whether the resulting graph remains connected. Be sure to add that vertex/edge back before doing the next deletion!

In directed graphs we are often concerned with *strongly connected components*, that is, partitioning the graph into chunks such that there are directed paths between all pairs of vertices within a given chunk. Road networks should be strongly connected, or else there will be places you can drive to but not drive home from without violating one-way signs.

The following idea enables us to identify the strongly connected components in a graph. It is easy to find a directed cycle using depth-first search, since any back edge plus the down path in the DFS tree gives such a cycle. All vertices in this cycle must be in the same strongly connected component. Thus we can shrink (contract) the vertices

on this cycle down to a single vertex representing the component, and then repeat. This process terminates when no directed cycle remains, and each vertex represents one strongly connected component.

10.1.3 Cycles in Graphs

All non-tree connected graphs contain cycles. Particularly interesting are cycles which visit all the edges or vertices of the graph.

An *Eulerian cycle* is a tour which visits every edge of the graph exactly once. The children's puzzle of drawing a geometric figure without ever lifting your pencil from the paper is an instance of finding an Eulerian cycle (or path), where the vertices are the junctions in the drawing and the edges represent the lines to trace. A mailman's route is ideally an Eulerian cycle, so he can visit every street (edge) in the neighborhood once before returning home. Strictly speaking, Eulerian cycles are *circuits*, not cycles, because they may visit vertices more than once.

An undirected graph contains an Eulerian cycle if it is connected and every vertex is of even degree. Why? The circuit must enter and exit every vertex it encounters, implying that all degrees must be even. This idea also suggests a way to find an Eulerian cycle, by building it one cycle at a time. We can find a simple cycle in the graph using the DFS-based algorithm discussed in Section 9.4.1. Deleting the edges on this cycle leaves each vertex with even degree. Once we have partitioned the edges into edge-disjoint cycles, we can merge these cycles arbitrarily at common vertices to build an Eulerian cycle.

In the case of directed graphs, the relevant condition is that all vertices have the same in-degree as out-degree. Peeling off any cycle preserves this property, and thus Eulerian cycles of directed graphs can be built in the same manner. *Eulerian paths* are tours that visit every edge exactly once but might not end up where they started from. These allow exactly two vertices to have parity violations, one of which must be the starting node and the other the ending node.

A *Hamiltonian cycle* is a tour which visits every vertex of the graph exactly once. The traveling salesman problem asks for the shortest such tour on a weighted graph. An Eulerian cycle problem in $G = (V, E)$ can be reduced to a Hamiltonian cycle problem by constructing a graph $G' = (V', E')$ such that each vertex in V' represents an edge of E and there are edges in E' connecting all neighboring pair of edges from G.

Unfortunately, no efficient algorithm exists for solving Hamiltonian cycle problems. Thus you have two options on encountering one. If the graph is sufficiently small, it can be solved via backtracking. Each Hamiltonian cycle is described by a permutation of the vertices. We backtrack whenever there does not exist an edge from the latest vertex to an unvisited one. If the graph is too large for such an attack we must try to find an alternate formulation of the problem, perhaps as an Eulerian cycle problem on a different graph.

10.1.4 Planar Graphs

Planar graphs are those which can be drawn in the plane such that no two edges cross each other. Many of the graphs we commonly encounter are planar. Every tree is planar: can you describe how to construct a non-crossing drawing for a given tree? Every road network in the absence of concrete/steel bridges must be planar. The adjacency structure of convex polyhedra also yield planar graphs.

Planar graphs have several important properties. First, there is a tight relation between the number of vertices n, edges m, and faces f of any planar graph. *Euler's formula* states that $n - m + f = 2$. Trees contain $n - 1$ edges, so any planar drawing of a tree has exactly one face, namely, the outside face. Any embedding of a cube (8 vertices and 12 edges) must contain six faces, as those who have played dice can attest to.

Efficient algorithms exist for testing the planarity of a graph and finding non-crossing embeddings, but all are fairly complicated to implement. Euler's formula gives an easy way to prove that certain graphs are *not* planar, however. Every planar graph contains at most $3n-6$ edges for $n > 2$. This bound means that every planar graph must contain a vertex of degree at most 5, and deleting this vertex leaves a smaller planar graph with this same property. Testing whether a given drawing is a planar embedding is the same as testing whether any of a given set of line segments intersect, which will be discussed when we get to geometric algorithms.

10.2 Minimum Spanning Trees

A *spanning tree* of a graph $G = (V, E)$ is a subset of edges from E forming a tree connecting all vertices of V. For edge-weighted graphs, we are particularly interested in the *minimum spanning tree*, the spanning tree whose sum of edge weights is the smallest possible.

Minimum spanning trees are the answer whenever we need to connect a set of points (representing cities, junctions, or other locations) by the smallest amount of roadway, wire, or pipe. Any tree is the smallest possible connected graph in terms of number of edges, while the minimum spanning tree is the smallest connected graph in terms of edge weight.

The two main algorithms for computing minimum spanning trees are Kruskal's and Prim's, and both are covered in most any algorithms course. We will present Prim's algorithm here because we think it is simpler to program, and because it gives us Dijkstra's shortest path algorithm with very minimal changes.

First, we must generalize the graph data structure from Chapter 9 to support edge-weighted graphs. Each edge-entry previously contained only the other endpoint of the given edge. We must replace this by a record allowing us to annotate the edge with weights:

```
typedef struct {
        int v;                          /* neighboring vertex */
```

```
        int weight;                     /* edge weight */
} edge;

typedef struct {
        edge edges[MAXV+1][MAXDEGREE];  /* adjacency info */
        int degree[MAXV+1];             /* outdegree of each vertex */
        int nvertices;                  /* number of vertices in graph */
        int nedges;                     /* number of edges in graph */
} graph;
```

and update the various initialization and traversal algorithms appropriately. This is not a complicated task.

Prim's algorithm grows the minimum spanning tree in stages starting from a given vertex. At each iteration, we add one new vertex into the spanning tree. A greedy algorithm suffices for correctness: we always add the lowest-weight edge linking a vertex in the tree to a vertex on the outside.

The simplest implementation of this idea would assign each vertex a Boolean variable denoting whether it is already in the tree (the array `intree` in the code below), and then searches all edges at each iteration to find the minimum weight edge with exactly one `intree` vertex.

Our implementation is somewhat smarter. It keeps track of the cheapest edge from any tree vertex to every non-tree vertex in the graph. The cheapest edge over all remaining non-tree vertices gets added in each iteration. We must update the costs of getting to the non-tree vertices after each insertion. However, since the new vertex is the only change in the tree all possible edge-weight updates come from its outgoing edges:

```
prim(graph *g, int start)
{
        int i,j;                        /* counters */
        bool intree[MAXV];              /* is vertex in the tree yet? */
        int distance[MAXV];             /* vertex distance from start */
        int v;                          /* current vertex to process */
        int w;                          /* candidate next vertex */
        int weight;                     /* edge weight */
        int dist;                       /* shortest current distance */

        for (i=1; i<=g->nvertices; i++) {
                intree[i] = FALSE;
                distance[i] = MAXINT;
                parent[i] = -1;
        }

        distance[start] = 0;
        v = start;
```

```
        while (intree[v] == FALSE) {
            intree[v] = TRUE;
            for (i=0; i<g->degree[v]; i++) {
                w = g->edges[v][i].v;
                weight = g->edges[v][i].weight;
                if ((distance[w] > weight) && (intree[w]==FALSE)) {
                        distance[w] = weight;
                        parent[w] = v;
                }
            }

            v = 1;
            dist = MAXINT;
            for (i=2; i<=g->nvertices; i++)
                if ((intree[i]==FALSE) && (dist > distance[i])) {
                        dist = distance[i];
                        v = i;
                }
        }
}
```

The minimum spanning tree itself or its cost can be reconstructed in two different ways. The simplest method would be to augment this procedure with statements that print the edges as they are found or total the weight of all selected edges in a variable for later return. Alternately, since the tree topology is encoded by the `parent` array it plus the original graph tells you everything about the minimum spanning tree.

This minimum spanning tree algorithm has several interesting properties which help solve many closely related problems:

- *Maximum Spanning Trees* — Suppose we hire an evil telephone company to connect a bunch of houses together, and that this company will be paid a price proportional to the amount of wire they install. Naturally, they will want to build as expensive a spanning tree as possible. The *maximum spanning tree* of any graph can be found by simply negating the weights of all edges and running Prim's algorithm. The most negative tree in the negated graph is the maximum spanning tree in the original.

  Most graph algorithms do not adapt so nicely to negative numbers. Indeed, shortest path algorithms have trouble with negative numbers, and certainly do *not* generate the longest possible path using this technique.

- *Minimum Product Spanning Trees* — Suppose we want the spanning tree with the smallest product of edge weights, assuming all edge weights are positive. Since $\lg(a \cdot b) = \lg(a) + \lg(b)$, the minimum spanning tree on a graph where each edge weight is replaced with its logarithm gives the minimum product spanning tree.